Prologos Language Grammar

A Literate Specification of the Surface Syntax

The Prologos Project — from Zachary Larson

2026-02-21

Contents

1 Introduction **3**

2 Lexical Grammar **3**
2.1 Comments 3
2.2 Identifiers 3
2.3 Numeric Literals 4
2.4 String Literals 5
2.5 Keyword Literals 5
2.6 Boolean Literals 5
2.7 Special Tokens 5
2.8 Bracket Types 5
2.9 Collection Literal Prefixes 6

3 Type Expressions **6**
3.1 Base Types 6
3.2 Parameterized Types 6
3.3 Function Types (Arrows) 7
3.4 Dependent Types (Angle Brackets) 7
3.5 Sigma Types (Product / Pair Types) 7
3.6 Equality Type 8
3.7 Union Types 8
3.8 Universe Levels 8
3.9 Type Holes 8
3.10 Typed Holes (Interactive Development) 8

4 Expressions **9**
4.1 Function Application . . . 9
4.2 Lambda Expressions . . . 9
4.3 Pattern Matching . . . 10
4.4 If (Conditional) . . . 11
4.5 Let (Local Binding) . . . 11
4.6 Pipe Operator (|>) . . . 12
4.7 Compose Operator (») . . . 12
4.8 Type Annotations (the) . . . 12
4.9 Pairs . . . 12
4.10 Quote and Quasiquote . . . 13
4.11 Collection Literals . . . 13
4.12 Partial Application . . . 14
4.13 Varargs . . . 14

5 Declarations **14**
5.1 Namespace (ns) . . . 14
5.2 Require . . . 15
5.3 Value Definition (def) . . . 15
5.4 Type Signature (spec) . . . 16
5.4.1 Extended Spec with Metadata . . . 16
5.5 Function Definition (defn) . . . 17
5.6 Algebraic Data Types (data) . . . 18
5.7 Traits and Implementations . . . 19
5.8 Property Declarations . . . 19
5.9 Functor Declarations (Named Type Abstractions) . . . 21
5.10 User-Defined Macros . . . 22

6 Multiplicity (QTT) **22**

7 Dependent Types and Eliminators **23**
7.1 Natural Number Elimination (natrec) . . . 23
7.2 Equality Elimination (J) . . . 23
7.3 Length-Indexed Vectors . . . 24

8 Full Program Example **24**

9 Appendix: S-Expression Mode **25**

10 Appendix: Whitespace Reader Rules **26**

11 Appendix: Reader Desugaring Table **26**

1 Introduction

Prologos is a functional-logic language unifying dependent types, session types, linear types (QTT), logic programming, and propagators. This document describes the *surface syntax* of the language as written in `.prologos` files.

Prologos has two syntactic modes:

1. **Whitespace-sensitive mode** (`.prologos` files): Indentation-based structure with `[]` for grouping and minimal punctuation. This is the primary mode.
2. **S-expression mode**: Parenthesized fallback for use in macros, tests, and when embedding in Racket. Every WS-mode form has a canonical sexp representation.

The grammar is organized bottom-up: lexical elements, then types, then expressions, then declarations.

> **Design Principle**: Prologos is *homoiconic* — code and data share the same representation. All syntactic sugar desugars to s-expressions. Macros operate on the post-parse representation, making code-as-data a first-class concept.

2 Lexical Grammar

2.1 Comments

Line comments begin with `;` and extend to end of line. There are no block comments.

```
;; This is a comment
def x : Nat zero  ; inline comment
```

2.2 Identifiers

Identifiers may contain letters, digits, hyphens, question marks, exclamation marks, and primes. They must not start with a digit.

```
identifier  ::= ident-start { ident-char }
ident-start ::= letter | '_' | '+' | '*' | '/' | '!' | '?' | '<' | '>' | '='
ident-char  ::= ident-start | digit | '-' | "'"
```

Naming conventions:

- Predicates use `?` suffix: `zero?`, `empty?`, `member?`
- Destructive operations use `!` suffix: `persist!`, `tvec-push!`
- Type names are capitalized: `Nat`, `List`, `Option`
- Value names are lowercase with hyphens: `sort-on`, `find-index`

Qualified names use `::` separator:

```
list::map       ;; qualified reference to list module's map
nat::add        ;; qualified reference to nat module's add
opt::unwrap-or  ;; qualified reference to option module's unwrap-or
```

2.3 Numeric Literals

Prologos has four numeric literal forms, plus a decimal variant for approximate literals:

```
nat-literal     ::= digit+ 'N'                (* Natural: 0N, 42N, 100N *)
int-literal     ::= ['-'] digit+              (* Integer: 42, -7, 0     *)
rat-literal     ::= ['-'] digit+ '/' digit+   (* Rational: 3/7, -1/3   *)
decimal-literal ::= ['-'] digit+ '.' digit+   (* Decimal: 3.14, 0.5    *)
approx-literal  ::= '~' (int | rat | decimal) (* Posit approx: ~42, ~3/7, ~3.14 *)
```

```
42          ;; Nat literal (Church-encoded natural - type infrastructure only)
42          ;; Int literal (arbitrary-precision integer)
3/7         ;; Rat literal (exact rational)
3.14        ;; Posit32 literal (approximate - same as ~3.14)
~3/7        ;; approximate Posit literal (from fraction)
~3.14       ;; approximate Posit literal (from decimal, stored as exact rational 157/5
```

Bare decimal literals (`3.14`, `0.5`) produce Posit32 values, the same as their tilde-prefixed equivalents (`~3.14`, `~0.5`). Internally, the decimal is stored as an exact rational (`157/50`) and encoded to the nearest Posit32 bit pattern.

Nat is intended for type-level infrastructure (indices, lengths, proofs), not general computation.

2.4 String Literals

Double-quoted with standard escape sequences (`\n`, `\t`, `\\`, `\"`).

```
"hello world"
"line one\nline two"
```

2.5 Keyword Literals

Keywords start with `:` and are used as map keys and enum-like values:

```
:name
:age
:hello
```

2.6 Boolean Literals

```
true
false
```

2.7 Special Tokens

Token	Meaning
`_`	Wildcard / type hole (inferred)
`_1 _2`	Numbered placeholders for partial application
`??`	Typed hole (interactive development)
`??name`	Named typed hole
`:0`	Erased multiplicity (use 0 times)
`:1`	Linear multiplicity (use exactly 1 time)
`:w`	Unrestricted multiplicity (use any times)
`zero`	Nat zero constructor
`unit`	Unit value
`refl`	Equality reflexivity proof

2.8 Bracket Types

Prologos uses four bracket types, each with distinct semantics:

Brackets	Purpose
`[...]`	Primary grouping: function application, params
`(...)`	Special forms: `(fn ...)`, `(match ...)`, types
`{...}`	Implicit type parameters, map literals
`<...>`	Dependent types, return type annotations

Inside any bracket pair, newlines are treated as whitespace (indentation is not significant).

2.9 Collection Literal Prefixes

Syntax	Type	Example
`'[...]`	`List`	`'[1N 2N 3N]`
`@[...]`	`PVec`	`@[1 2 3]`
`~[...]`	`LSeq`	`~[1 2 3]`
`#{...}`	`Set`	`#{1 2 3}`
`{k v}`	`Map`	`{:name "Ada"}`

3 Type Expressions

In a dependently-typed language, types and terms share the same expression syntax. This section highlights type-specific forms.

3.1 Base Types

```
base-type ::= 'Type'     (* universe of types, level inferred *)
            | 'Nat'      (* natural numbers                   *)
            | 'Int'      (* arbitrary-precision integers       *)
            | 'Rat'      (* exact rationals                    *)
            | 'Bool'     (* booleans                           *)
            | 'Unit'     (* unit type                          *)
            | 'Symbol'   (* symbols for homoiconicity          *)
            | 'Keyword'  (* keyword type for map keys          *)
            | 'Datum'    (* code-as-data algebraic type        *)
```

```
def x : Nat zero
def b : Bool true
def u : Unit unit
```

3.2 Parameterized Types

Types are applied to arguments via juxtaposition in brackets:

```
List Nat            ;; list of natural numbers
Option Bool         ;; optional boolean
Map Keyword Int     ;; keyword-to-int map
Vec Nat 3N          ;; length-3 vector of Nats (dependent!)
```

```
Set Int             ;; set of ints
PVec Nat            ;; persistent vector of Nats
```

3.3 Function Types (Arrows)

Arrows are right-associative infix operators:

```
arrow-type ::= type '->' type    (* unrestricted function *)
             | type '-0>' type   (* erased function       *)
             | type '-1>' type   (* linear function       *)
             | type '-w>' type   (* unrestricted, explicit *)

Nat -> Nat                ;; simple function
Nat -> Nat -> Bool        ;; curried: Nat -> (Nat -> Bool)
[List A] -> Nat           ;; bracketed domain
[A -> B] -> [List A] -> List B  ;; higher-order
```

3.4 Dependent Types (Angle Brackets)

Angle brackets delimit dependent Pi (universal) and Sigma (existential) types:

```
dependent-type ::= '<' '(' ident ':' type ')' '->' type '>'  (* dependent Pi   *)
                 | '<' '(' ident ':' type ')' '*'  type '>'  (* dependent Sigma *)
                 | '<' ident ':' type '->' type '>'           (* shorthand Pi   *)

;; Dependent function: "for all n : Nat, a vector of length n"
<(n : Nat) -> Vec Nat n>

;; Dependent pair: "a natural n together with a Vec of that length"
<(n : Nat) * Vec Nat n>

;; Shorthand (single binder, no inner parens):
<n : Nat -> Vec Nat n>
```

3.5 Sigma Types (Product / Pair Types)

Non-dependent pair types use infix *:

```
;; Non-dependent pair
Nat * Bool
```

```
;; In return type of split-at:
[Sigma [_ <List A>] [List A]]

;; Dependent Sigma via angle brackets:
<(x : Nat) * Vec Nat x>
```

3.6 Equality Type

```
;; Eq type : proof that two terms are equal
(Eq Nat zero zero)       ;; 0 = 0 at type Nat
(Eq Nat [add x y] [add y x])  ;; commutativity
```

3.7 Union Types

Union types use infix `|`:

```
Nat | Bool         ;; either a Nat or a Bool
Int | Rat | Nat    ;; right-associative: Int | (Rat | Nat)
```

3.8 Universe Levels

```
Type           ;; universe, level inferred
(Type 0)       ;; explicit level 0
(Type 1)       ;; explicit level 1 (contains Type 0)
```

3.9 Type Holes

The wildcard `_` stands for an inferred type:

```
def x : _ zero    ;; type inferred as Nat
map _ xs          ;; type argument inferred
```

3.10 Typed Holes (Interactive Development)

The `??` token is a *typed hole* — distinct from `_` (inference hole). Where `_` means "infer this for me silently," `??` means "I don't know what goes here — show me what's possible." The type checker produces a diagnostic report with the expected type, local context, and suggestions.

```
defn reverse
  | [nil]        -> nil
  | [[cons h t]] -> ??
  ;; Report: Hole ?? : [List A]
  ;;   h : A, t : [List A], reverse : [List A] -> [List A]
```

Named holes label specific incomplete positions:

```
defn zip-with [f xs ys]
  match [xs ys]
    | [[cons x xs'] [cons y ys']] -> [cons ??combine ??recurse]
```

`??` is the syntactic foundation for Editor-Assisted Interactive Development (Agda/Idris-style hole-driven programming). See the Extended Spec Design document for the full vision.

4 Expressions

4.1 Function Application

Application is juxtaposition, with brackets `[]` for grouping:

```
;; Simple application
suc zero                    ;; suc(zero)

;; Multi-argument (uncurried convention)
add x y                     ;; add(x, y)

;; Brackets for grouping
[add x y]                   ;; same as add x y
cons [f a] [map f as]       ;; cons(f(a), map(f, as))

;; Nested application
map [add 1N _] '[1N 2N 3N]  ;; map with partial application
```

4.2 Lambda Expressions

```
lambda ::= 'fn' binder+ body
         | '(' 'fn' '(' ident ':' type ')' body ')'   (* sexp mode *)
```

```
;; WS mode: fn with bracket binder
fn [x : Nat] [suc x]

;; Multiple binder groups
fn [x : Nat] [y : Nat] [add x y]

;; Bare parameter (type inferred from context)
fn x [suc x]

;; Sexp mode: explicit binder
(fn (x : Nat) (suc x))

;; With multiplicity annotation
(fn (x :1 Nat) x)          ;; linear lambda
(fn (x :0 Nat) zero)       ;; erased lambda (can't use x)
```

4.3 Pattern Matching

`match` is the primary pattern matching form, using `|` for arm separation:

```
match-expr ::= 'match' scrutinee match-arm+
match-arm  ::= '|' pattern '->' body

;; Match on Nat
match n
  | zero  -> zero
  | suc k -> add k k

;; Match on Bool
match b
  | true  -> suc zero
  | false -> zero

;; Match on List
match xs
  | nil       -> zero
  | cons a as -> suc [length as]

;; Match on Option
match [find pred xs]
```

```
  | none   -> default
  | some v -> v

;; Nested match
match xs
  | nil       -> nil
  | cons a as -> match [pred a]
    | true  -> cons a [filter pred as]
    | false -> filter pred as
```

Available patterns:

- **Variable**: `x`, `n`, `rest` – binds the matched value
- **Wildcard**: `_` – matches anything, discards
- **Constructor**: `zero`, `suc k`, `nil`, `cons a as`, `true`, `false`, `none`, `some v`, `ok v`, `err e`, `datum-nil`, `datum-cons h t`, etc.
- **Nested**: `[cons x nil]` – grouped pattern in brackets

4.4 If (Conditional)

`if` is a macro that desugars to `match` on Bool:

```
;; Three-argument if
if [zero? n] base-case [step n]

;; Equivalent match:
match [zero? n]
  | true  -> base-case
  | false -> step n
```

4.5 Let (Local Binding)

`let` is a macro that desugars to application:

```
;; Single binding
let x = [add 1N 2N]
  suc x

;; With type annotation
let x : Nat = [add 1N 2N]
  suc x
```

4.6 Pipe Operator (|>)

The pipe operator threads a value left-to-right through functions:

```
;; Binary pipe
zero |> suc |> suc |> suc    ;; = suc(suc(suc(zero))) = 3

;; Block pipe with indented steps
|> nums
  map suc-fn
  filter is-positive
  reduce sum-rf zero

;; Block pipes support automatic loop fusion:
;; consecutive map/filter steps fuse into a single-pass
;; transducer when followed by a terminal (reduce, sum, etc.)
```

4.7 Compose Operator (»)

Left-to-right function composition:

```
;; Compose two functions
[suc >> suc] zero           ;; = suc(suc(zero)) = 2

;; Pipe into a composed function
zero |> [suc >> double]     ;; = double(suc(zero))
```

4.8 Type Annotations (the)

Explicit type annotation on an expression:

```
;; Annotate with explicit type
(the Nat zero)
(the [List Nat] nil)
```

4.9 Pairs

```
;; Construct a pair
pair 1N true

;; Project from a pair
first [pair 1N true]        ;; = 1N
```

```
second [pair 1N true]       ;; = true

;; Aliases: fst, snd
fst [pair x y]
snd [pair x y]
```

4.10 Quote and Quasiquote

Quote ' converts code to `Datum` (code-as-data):

```
'foo          ;; -> datum-sym
'42           ;; -> datum-nat
':hello       ;; -> datum-kw
'true         ;; -> datum-bool
'()           ;; -> datum-nil
'(add 1 2)   ;; -> datum-cons chain

;; Quasiquote with unquote
def val : Datum [datum-nat 10]
`,val          ;; splice val into template
`(add 1 ,val)  ;; template with hole
```

4.11 Collection Literals

```
;; List literal (linked list)
'[1N 2N 3N]

;; List with tail
'[1N 2N | rest-list]

;; PVec (persistent vector)
@[1 2 3 4 5]

;; Set
#{1 2 3}

;; Lazy sequence
~[1 2 3]

;; Map (key-value pairs)
{:name "Ada" :age 36}
```

```
;; Mixed-type map: value type auto-inferred as union
;; {:name "Alice" :age zero} : Map Keyword (Nat | String)
;; map-assoc widens: (map-assoc nat-map :k "s") : Map Keyword (Nat | String)
{:name "Alice" :age zero}
```

4.12 Partial Application

Numbered placeholders `_1`, `_2` enable positional reordering:

```
;; Wildcard _ fills rightmost position
[add 1N _]           ;; fn x -> add 1N x

;; Numbered holes for reordering
[div _2 _1]          ;; fn x y -> div y x
```

4.13 Varargs

The `...` marker in specs indicates variadic arguments. In `defn`, `...name` collects excess arguments into a `List`:

```
spec sum-all Nat ... -> Nat
defn sum-all [...xs]
  sum xs

sum-all 1 2 3          ;; = 6
sum-all 1 2 3 4 5      ;; = 15

;; Mixed fixed + varargs
spec first-plus-rest Nat Nat ... -> Nat
defn first-plus-rest [first ...rest]
  add first [sum rest]

first-plus-rest 100 1 2 3  ;; = 106
```

5 Declarations

5.1 Namespace (ns)

Must be the first form in a file. Controls module identity and prelude loading:

```
;; Standard namespace: auto-imports prelude
ns my-project.utils

;; Bare namespace: no prelude
ns prologos.data.list :no-prelude
```

The prelude automatically provides: `Nat`, `Bool`, `List`, `Option`, `Result`, `Pair` operations, `Eq=/=Ord=/=Add=/=Sub=/=Mul=/=Neg=/=Abs=/=FromInt=/=Num=/=Fractional` traits and instances.

5.2 Require

Import modules with named or aliased references:

```
;; Named imports
require [prologos.data.list :refer [List nil cons map length]]

;; Aliased module
require [prologos.data.nat :as nat :refer [add mult]]

;; Multiple requires
require [prologos.data.list   :refer [List nil cons map]]
        [prologos.data.nat    :refer [add mult]]
        [prologos.data.option :refer [Option none some]]
```

5.3 Value Definition (def)

```
;; With type annotation
def x : Nat zero

;; Shorthand with :=
def x := zero

;; Multi-line body (indented)
def nums : [List Nat]
  cons 1N [cons 2N [cons 3N [nil Nat]]]

;; Private (not auto-exported)
def- internal-state : Nat zero
```

5.4 Type Signature (spec)

`spec` declares a type signature separately from the definition:

```
;; Simple function type
spec factorial Nat -> Nat

;; With implicit type parameters
spec map {A B : Type} [A -> B] [List A] -> List B

;; With trait constraints
spec elem {A : Type} [Eq A] A [List A] -> Bool

;; With varargs
spec sum-all Nat ... -> Nat

;; With multiple implicits and constraints
spec sort-on {A B : Type} [B -> B -> Bool] [A -> B] [List A] -> List A
```

5.4.1 Extended Spec with Metadata

`spec` optionally accepts a trailing metadata map using the implicit map syntax. Keyword-headed children after the type signature are collected into a metadata hash. Recognized keys include `:implicits`, `:where`, `:doc`, `:examples`, `:properties`, `:see-also`, `:pre`, `:post`, `:refines`. Unrecognized keys are stored but not acted upon (forward-compatible).

The `:implicits` key lifts implicit binders out of the type signature into metadata, freeing the signature to express pure function shape. Inline `{A : Type}` and `:implicits` coexist; both are merged.

```
;; With :implicits - clean signature:
spec sort [List A] -> [List A]
  :implicits {A : Type}
  :where (Ord A)
  :doc "Sorts a list in ascending order"
  :examples
    - [sort '[3N 1N 2N]] => '[1N 2N 3N]
    - [sort '[]] => '[]
  :properties (sortable-laws A)
  :see-also [reverse filter]
```

```
;; HKT case - :implicits dramatically improves readability:
spec gmap [A -> B] -> [C A] -> [C B]
  :implicits {A B : Type} {C : Type -> Type}
  :where (Seqable C) (Buildable C)

;; With functor - Pi types hidden entirely:
spec xf-compose [Xf A B] -> [Xf B C] -> [Xf A C]
  :implicits {A B C : Type}
  :properties (transducer-fusion-laws A B C)
```

The metadata is entirely optional. A simple `spec add Nat Nat -> Nat` is unchanged. Metadata keys follow the principle of progressive disclosure: add `:doc` and `:examples` for public APIs, `:properties` for libraries, `:pre=`/:post= for safety-critical code, `:refines` for verified code.

Each `:properties` entry is a map with three keys:

Key	Meaning
`:name`	Human-readable label for the property
`:forall`	Universally quantified variables (binders)
`:holds`	Boolean expression that must be true

See the Extended Spec Design document (`docs/tracking/2026-02-22_EXTENDED_SPEC_DESIGN.org` for the full research survey and design rationale.

5.5 Function Definition (defn)

```
;; Basic function
spec add Nat -> Nat -> Nat
defn add [x y]
  match x
    | zero  -> y
    | suc n -> suc [add n y]

;; With inline type
defn suc-all [xs : List Nat] : List Nat
  map [add _ 1N] xs

;; With return type annotation
impl Eq Nat
  defn eq? [x y] <Bool>
    nat-eq? x y
```

```
;; Multi-arity with |
defn fact
  | [zero]  -> suc zero
  | [suc n] -> mult [suc n] [fact n]

;; Private
defn- helper [x]
  suc x
```

5.6 Algebraic Data Types (data)

```
;; Enumeration (nullary constructors)
data Bool
  true
  false

;; Parameterized with constructors
data List {A}
  nil
  cons : A -> List A

;; Multiple implicit params
data Either {A B}
  left  : A
  right : B

;; Single constructor (wrapper)
data Ordering
  lt-ord
  eq-ord
  gt-ord

;; Private data type
data- InternalTree {A}
  leaf
  node : A -> InternalTree A -> InternalTree A
```

5.7 Traits and Implementations

```
;; Trait definition
trait Eq {A}
  eq? : A -> A -> Bool

;; Implementation
impl Eq Nat
  defn eq? [x y] <Bool>
    nat-eq? x y

;; Multi-method trait
trait Ord {A}
  compare : A -> A -> Ordering

;; Generic functions using trait dicts
spec eq-neq [Eq A] A A -> Bool
defn eq-neq [dict x y]
  not [dict x y]

;; Bundle: named trait combination
bundle Num := (Add, Sub, Mul, Neg, Abs, FromInt)

;; Trait with :laws reference
trait Functor {F : Type -> Type}
  :laws (functor-laws F)
  fmap : {A B : Type} [A -> B] -> [F A] -> [F B]
```

5.8 Property Declarations

A `property` declares a named, composable group of propositions — analogous to `bundle` for traits. Properties compose via `:includes` (conjunction) and attach to specs via `:properties` or to traits via `:laws`.

```
;; Basic property declaration
property sortable-laws {A : Type}
  :where (Ord A)
  - :name "idempotent"
    :forall {xs : [List A]}
    :holds [eq? [sort [sort xs]] [sort xs]]
  - :name "length-preserving"
```

```
    :forall {xs : [List A]}
    :holds [eq? [length [sort xs]] [length xs]]

;; Composition via :includes (like bundle for traits)
property monoid-laws {A : Type}
  :where (Add A) (AdditiveIdentity A)
  :includes (semigroup-laws A)
  - :name "left-identity"
    :forall {x : A}
    :holds [eq? [add additive-identity x] x]
  - :name "right-identity"
    :forall {x : A}
    :holds [eq? [add x additive-identity] x]

;; Hierarchical: monad includes applicative includes functor
property monad-laws {M : Type -> Type}
  :where (Monad M)
  :includes (applicative-laws M)
  - :name "left-identity"
    :forall {a : A} {f : [A -> [M B]]}
    :holds [eq? [bind [pure a] f] [f a]]

;; Usage in spec
spec sort [List A] -> [List A]
  :implicits {A : Type}
  :where (Ord A)
  :properties (sortable-laws A)

;; Higher-order: parameterized over functions
property sorting-properties {A : Type} {f : [List A] -> [List A]}
  :where (Ord A)
  - :name "idempotent"
    :forall {xs : [List A]}
    :holds [eq? [f [f xs]] [f xs]]

spec sort [List A] -> [List A]
  :where (Ord A)
  :properties (sorting-properties A sort)
```

Property clause names are scoped to their declaring block. When `:includes`

flattens a hierarchy, names are qualified with `/`: `functor-laws/identity`, `monad-laws/left-identity`, etc.

5.9 Functor Declarations (Named Type Abstractions)

A `functor` declares a named, parameterized type abstraction with optional category-theoretic metadata. Transparent by default — the type checker unfolds to the `:unfolds` form during elaboration. Metadata keys map CT concepts to approachable language.

```
;; Simple type synonym
functor FilePath
  :unfolds String

;; Parameterized synonym with documentation
functor Result {A}
  :unfolds [Either String A]
  :doc "A computation that may fail with a string error"

;; Full category-theoretic treatment
functor Xf {A B : Type}
  :doc "A transducer: transforms A-reductions into B-reductions"
  :compose xf-compose
  :identity id-xf
  :laws (transducer-fusion-laws A B)
  :unfolds <(S :0 Type) -> [S -> B -> S] -> S -> A -> S>

;; Optics
functor Lens {S T A B : Type}
  :doc "A bidirectional accessor: view and update a part of a structure"
  :compose lens-compose
  :identity lens-id
  :laws (lens-laws S T A B)
  :see-also [Prism Traversal Iso]
  :unfolds <{F : Type -> Type} -> (Functor F) -> [A -> [F B]] -> S -> [F T]>

;; Usage in specs - the functor name replaces raw Pi types:
spec xf-compose [Xf A B] -> [Xf B C] -> [Xf A C]
  :implicits {A B C : Type}
spec into-list [Xf A B] -> [List A] -> [List B]
```

```
:implicits {A B : Type}
```

CT concept	Key	Meaning in plain English
Object mapping	`:unfolds`	"What this expands to under the hood"
Morphism composition	`:compose`	"How to chain two of these together"
Identity morphism	`:identity`	"The do-nothing version"
Laws	`:laws`	"What rules these always follow"

Only `:unfolds` is required. The rest are progressive.

5.10 User-Defined Macros

```
;; Simple macro: double-it x --> add x x
defmacro double-it [$x] [add $x $x]

double-it 5             ;; = 10

;; Conditional macro
defmacro when-nonzero [$n $body]
  [if [not [zero? $n]] $body zero]

when-nonzero 5 [double 5]    ;; = 10
when-nonzero zero [double 5]  ;; = 0

;; Introspection
(expand (double-it 5))        ;; shows expansion
(expand-1 (double-it 5))      ;; one step
(expand-full (when true [suc zero]))  ;; all steps with labels
```

6 Multiplicity (QTT)

Prologos uses Quantitative Type Theory for resource tracking. Each variable binding has a *multiplicity* annotation:

Annotation	Meaning	Syntax
`:0`	Erased (0 uses at runtime)	`(x :0 Nat)`
`:1`	Linear (exactly 1 use)	`(x :1 Nat)`
`:w`	Unrestricted (any uses)	`(x :w Nat)`

Arrow types carry multiplicities:

```
Nat -> Nat            ;; unrestricted (default :w)
Nat -0> Nat           ;; erased: argument not used at runtime
Nat -1> Nat           ;; linear: argument used exactly once
Nat -w> Nat           ;; unrestricted: explicit

;; Erased argument (type-level only)
(fn (A :0 Type) (fn (x :w A) x))     ;; polymorphic identity

;; Linear function
(fn (x :1 Nat) x)                    ;; must use x exactly once
```

7 Dependent Types and Eliminators

7.1 Natural Number Elimination (natrec)

The low-level eliminator for natural numbers:

```
;; natrec motive base step target
;; motive : Nat -> Type
;; base   : motive zero
;; step   : (n : Nat) -> motive n -> motive (suc n)
;; target : Nat
natrec motive base step target
```

In practice, `match` is preferred:

```
;; Idiomatic: match instead of natrec
defn double [n]
  match n
    | zero  -> zero
    | suc k -> suc [suc [double k]]
```

7.2 Equality Elimination (J)

```
;; J motive base left right proof
;; Given proof : Eq A left right
;; J computes: motive(left, right, proof) via base(left)
J motive base left right refl
```

7.3 Length-Indexed Vectors

```
;; Vec A n : a vector of exactly n elements of type A
;; Fin n   : a number guaranteed less than n

vnil Nat                          ;; empty: Vec Nat 0
vcons Nat 2N x xs                 ;; cons:  Vec Nat 3 (given xs : Vec Nat 2)
vhead Nat 2N v                    ;; safe head (non-empty guaranteed)
vtail Nat 2N v                    ;; safe tail
vindex Nat 3N [fzero 2N] v       ;; safe index via Fin
```

8 Full Program Example

A complete Prologos program demonstrating multiple features:

```
ns examples.demo

require [prologos.data.list   :refer [List nil cons map reduce sum length]]
        [prologos.data.nat    :refer [add mult]]
        [prologos.data.option :refer [Option none some]]

;; Type signature with implicit params
spec factorial Nat -> Nat
defn factorial [n]
  match n
    | zero  -> suc zero
    | suc k -> mult n [factorial k]

;; Higher-order function with trait constraint
spec sum-mapped {A : Type} [A -> Nat] [List A] -> Nat
defn sum-mapped [f xs]
  sum [map f xs]

;; Data type definition
data Tree {A}
  leaf
  node : A -> Tree A -> Tree A

;; Pattern matching on user-defined types
spec tree-size [Tree A] -> Nat
```

```
defn tree-size [t]
  match t
    | leaf       -> zero
    | node _ l r -> suc [add [tree-size l] [tree-size r]]

;; Using pipe operator
def result : Nat
  |> '[1N 2N 3N 4N 5N]
    map [add 1N _]
    reduce add 0N

;; Macro definition
defmacro unless [$cond $body] [if $cond zero $body]

;; Using the macro
unless [zero? 5] [factorial 5]
```

9 Appendix: S-Expression Mode

Every WS-mode form has a canonical s-expression representation. In sexp mode, all grouping uses parentheses and whitespace is not significant:

```
;; WS mode:
defn add [x y]
  match x
    | zero  -> y
    | suc n -> suc [add n y]

;; Equivalent sexp mode:
(defn add (x y)
  (match x
    (zero  -> y)
    (suc n -> (suc (add n y)))))

;; WS mode:
spec map {A B : Type} [A -> B] [List A] -> List B

;; Equivalent sexp mode:
(spec map {A B : Type} (-> (-> A B) (-> (List A) (List B))))
```

10 Appendix: Whitespace Reader Rules

The WS reader converts indentation to explicit structure:

1. **Same indentation** as previous line: NEWLINE token (sibling separator)
2. **Deeper indentation**: INDENT token (child block begins)
3. **Shallower indentation**: DEDENT token(s) (child blocks end)
4. **Inside brackets** `[]`, `()`, `{}`, `<>`: newlines are whitespace
5. **Blank/comment lines**: ignored for indentation purposes
6. **Column 0**: always starts a new top-level form

11 Appendix: Reader Desugaring Table

Surface syntax	Reader output	Meaning
`'expr`	`($quote expr)`	Quote to Datum
`` `expr ``	`($quasiquote expr)`	Quasiquote template
`,expr`	`($unquote expr)`	Unquote (splice into QQ)
`42N`	`($nat-literal 42)`	Natural number literal
`~42`	`($approx-literal 42)`	Approximate (Posit) literal
`3.14`	`($decimal-literal 157/50)`	Bare decimal → Posit32
`:name`	`keyword token`	Keyword literal
`'[1 2 3]`	`($list-literal 1 2 3)`	List literal
`@[1 2 3]`	`($vec-literal 1 2 3)`	PVec literal
`~[1 2 3]`	`($lseq-literal 1 2 3)`	Lazy sequence literal
`#{1 2 3}`	`($set-literal 1 2 3)`	Set literal
`...name`	`($rest-param name)`	Rest/vararg parameter
`x \vert> f`	`($pipe-gt x f)`	Pipe left-to-right
`f >> g`	`($compose f g)`	Compose left-to-right